Football Forecaster — Project Knowledge Notes

> My personal reference for understanding what I built and why.
> Updated as I learn. Don't let this replace understanding — use it to reinforce it.

The Stack — What Each Part Does

PostgreSQL

- The actual database where all data lives
- Stores: leagues, teams, matches, odds, predictions, paper bets
- Speaks SQL — cannot talk directly to Python

SQLAlchemy

- The **translator** between Python and PostgreSQL
- Python speaks Python. PostgreSQL speaks SQL. SQLAlchemy sits in the middle.
- Analogy: PostgreSQL = storage room, FastAPI = receptionist, SQLAlchemy = staff member who fetches/saves data
- Key components used in this project:
  - `create_engine()` — the **phone line/road** to the database (holds connection details)
  - `sessionmaker()` — the **car/template** for how conversations with the DB happen
  - `Session` — one actual conversation with the database

FastAPI

- The API gateway — receives requests from the outside world (frontend, tools, etc.)
- Defines routes like `/predictions/generate`, `/fixtures/upcoming`
- Handles request/response automatically

---

db.py — How the Database Connection Works

```python
```

```python
engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

engine

- Creates the **road** between Python and PostgreSQL
- Holds the address, username, password
- `pool_pre_ping=True` = checks if connection is still alive before using it

SessionLocal

- A **template** for database sessions
- Defines the rules: don't auto-commit, don't auto-flush

get_db()

- Opens a session, hands it out, then closes it when done
- Uses `yield` not `return` — **this is critical**

yield vs return

`return`	`yield`
Gives value and STOPS	Gives value and PAUSES
Function is done forever	Function resumes after caller finishes
`finally` block may not run	`finally` ALWAYS runs

Why yield matters here: If we used `return`, `db.close()` would never run. Hundreds of open DB connections would build up and crash the app. `yield` guarantees cleanup even if the route throws an error.

Dependency Injection — What `Depends` Does

```python
def upcoming_fixtures(league: str = "EPL", db: Session = Depends(get_db)):
```

- `Depends(get_db)` tells FastAPI: *"Before running this route, call get_db() and pass the result in as db"*
- You never call `get_db()` yourself — FastAPI does it automatically every time the route is hit
- This is called **Dependency Injection** — your route doesn't care how the session is made, it just receives a working one

Analogy: Chef (route) needs a chopping board (db session). Kitchen assistant (get_db) always prepares it before the chef starts and cleans it up after. `Depends` is the rule that enforces this.

.mappings().all() — Why We Use It

```python
rows = db.execute(q, {"league": league}).mappings().all()
```

`.all()` alone	`.mappings().all()`
Returns Row objects (like tuples)	Returns dictionary-like objects
Access by position: `row[0]`, `row[2]`	Access by name: `row["home_team"]`
Easy to get wrong, hard to read	Clear, self-documenting

Always use `.mappings().all()` — your future self will thank you when debugging.

The Poisson Model — How Predictions Are Made

What is Lambda (lam)?

- `lam` = **expected number of goals** a team will score in a match
- NOT actual goals — it's a prediction based on past performance
- Example: `lam_home = 1.4` means home team is expected to score 1.4 goals

poisson_pmf(k, lam)

```python
def poisson_pmf(k: int, lam: float) -> float:
    return math.exp(-lam) * (lam ** k) / math.factorial(k)
```

- **PMF = Probability Mass Function**
- Answers: *"Given expected goals of lam, what is the probability of scoring EXACTLY k goals?"*
- Example with `lam = 1.5`:
  - `poisson_pmf(0, 1.5)` = P(score exactly 0)
  - `poisson_pmf(1, 1.5)` = P(score exactly 1)
  - `poisson_pmf(2, 1.5)` = P(score exactly 2)

poisson_cdf(k, lam)

- **CDF = Cumulative Distribution Function**
- Answers: *"What is the probability of scoring k OR FEWER goals?"*
- `poisson_cdf(2, lam)` = P(0 goals) + P(1 goal) + P(2 goals)

The Complement Rule

```python
return 1.0 - poisson_cdf(k, lam_total)
```

- For **Over 2.5** to win → need 3 or more goals
- `poisson_cdf(2, lam)` gives P(2 or fewer)
- `1 - P(2 or fewer)` = **P(3 or more)** ← what we want
- Rule: P(event) + P(not event) = 1, so P(event) = 1 - P(not event)
- **This is called the complement rule** — one of the most used tricks in probability

Real Example

```
lam_home = 1.4, lam_away = 1.1
lam_total = 2.5
poisson_cdf(2, 2.5) ≈ 0.54  → 54% chance of 2 or fewer goals
1 - 0.54 = 0.46             → 46% chance of Over 2.5 ✅
```

Why lam_home + lam_away?

- Home and away goals are **independent** of each other
- Total goals = home goals + away goals
- When two independent Poisson variables are added → their lambdas add too
- So total goal expectation = `lam_home + lam_away`

Key Bugs Found in the Project (Fix These)

Bug 1 — Dead code in pricing.py

- `mock_model_probability()` with hardcoded values still exists
- Your real Poisson model replaced it — this function is never used
- **Fix:** Delete it or clearly mark it as deprecated

Bug 2 — Missing odds_decimal column (BREAKING)

- `paper_bets/place` queries `p.odds_decimal` from predictions table
- But `predictions` table has NO `odds_decimal` column — only `implied_probability`
- **Fix:** Either add `odds_decimal` to predictions table, or calculate it as `1/implied_probability` in the query

Bug 3 — model_version mismatch

- `/paper-bets/place` defaults to `model_version = "mock_v1"`
- But real predictions are saved under `"poisson_v1"`
- Paper bets will always place 0 bets because nothing matches
- **Fix:** Change default to `"poisson_v1"`

---

estimate_lambdas() — The Brain of the Model

Attack/Defence Strength Modelling

```python
lam_home = (home_scored + away_conceded) / 2.0
lam_away = (away_scored + home_conceded) / 2.0
```

- **Never use home_scored alone** — it ignores who they scored against

- A team that scored 3 goals against a weak defence looks better than they are
- Combining attack + opponent defence gives a fairer picture
- Technical name: **Attack/Defence Strength Modelling**

Shrinkage — Protecting Against Small Samples

```python
lam_home = (lam_home + league_home_base) / 2.0
```

- Problem: A team with only 2 games of data can have extreme/unreliable numbers
- Solution: Average their estimate with the league average to pull it back toward reality
- Example: Team averaging 4.0 goals from 2 games → nudged to (4.0 + 1.35) / 2 = 2.675
- Technical name: **Shrinkage** — pulling extreme estimates toward the mean
- You'll see this again in ML as **regularisation**

Clamping — Preventing Impossible Outputs

```python
lam_home = max(0.2, min(lam_home, 3.5))
```

- Hard limits: lambda can never go below 0.2 or above 3.5
- Pure sanity check — no negative expected goals, no absurd predictions

The Full Intelligence Chain

1. Use last 5 matches before kickoff (no leakage)
2. Combine attack strength + opponent defence weakness
3. Shrink toward league baseline if sample is small
4. Clamp to sane bounds
5. Feed into Poisson PMF/CDF to get probabilities

Edge and Expected Value (EV)

Edge

```python
def compute_edge(model_probability, implied_probability):
    return model_probability - implied_probability
```

- **Edge = your model's probability minus the bookmaker's implied probability**
- Example: Your model says 60% chance, bookmaker implies 50% → Edge = 10%
- **Positive edge = your model is beating the bookmaker**
- That gap is where profit lives — finding positive edge bets consistently is value betting

Implied Probability

```python
def implied_probability_from_odds(odds_decimal):
    return 1.0 / odds_decimal
```

- Bookmaker odds of 2.0 → implied probability = 1/2.0 = 50%
- This is what the bookmaker thinks the probability is (plus their margin)

Expected Value (EV)

```python
profit_if_win = stake * (odds_decimal - 1.0)
return (model_probability * profit_if_win) - ((1.0 - model_probability) * stake)
```

- **EV translates edge into rands and cents**
- Formula: `(P(win) × profit) - (P(lose) × stake)`

Real example:

```
Stake = R10, Odds = 2.0, Model says 60% win chance
profit_if_win = R10 × (2.0 - 1.0) = R10
EV = (0.60 × R10) - (0.40 × R10)
EV = R6 - R4 = +R2
```

- +R2 EV means on average you make R2 every time you place this bet
- Not guaranteed on one bet — but over hundreds of bets, positive EV = profit
- **Negative EV = losing money on average** (every casino bet you ever make)
- Your app finds bets where YOU have positive EV because your model is smarter than the bookmaker's odds

The Golden Rule

- Don't bet every match — only bet when `edge > 0` and `EV > threshold`
- This is what your `min_ev` and `min_edge` filters in your routes do

Paper Betting System — How It Works

What is a Paper Bet?

- A **simulated bet** — recorded as if placed with real money, but no actual money changes hands
- Purpose: test if your model makes money **before** risking anything real
- Like a flight simulator before flying a real plane
- Technical name: **backtesting/forward testing** — how every serious quant strategy is validated

The 4 Routes and What They Do

```
POST /paper-bets/place    → place simulated bets on positive EV predictions
POST /paper-bets/settle   → grade them after matches finish
GET  /paper-bets/summary  → overall stats (ROI, win rate, avg edge)
GET  /paper-bets/equity   → profit curve over time
```

Duplicate Guard in /place

```python
AND NOT EXISTS (
    SELECT 1 FROM paper_bets pb
    WHERE pb.match_id = p.match_id
    AND pb.market = p.market
)
```

- Prevents placing the same bet twice if you run `/place` multiple times
- Without this: double bets, fake profit numbers, meaningless ROI
- Pattern: only place if this match + market combo doesn't already exist

Idempotent Guards in /settle

```python
WHERE pb.settled_at IS NULL     -- only process unsettled bets
...
settled_at = now                -- mark as settled so it's skipped next time
```

- `settled_at IS NULL` = not yet settled → process it

- `settled_at = now` = mark as done → skipped on next run
- **Idempotent** = running something multiple times produces the same result as once
- Both `/place` and `/settle` are idempotent — safe to run repeatedly

ROI vs Win Rate — Which Matters More

ROI matters more than win rate.

A model can win 70% of bets and STILL lose money:

```
7 wins at odds 1.20 → 7 × R2 profit  = +R14
3 losses at odds 1.20 → 3 × R10 loss = -R30
Net = -R16 ← LOSING despite 70% win rate
```

A model can win only 30% and make money:

```
3 wins at odds 4.0 → 3 × R30 profit = +R90
7 losses at odds 4.0 → 7 × R10 loss = -R70
Net = +R20 ← PROFITABLE despite 30% win rate
```

Win rate is vanity. ROI is sanity. This is why your app calculates EV and edge — not just win/loss.

The Equity Curve

```python
equity = 0
for r in rows:
    equity += float(r["daily_profit"])
    curve.append({"date": r["day"], "equity": equity})
```

- Accumulates daily profit to show total profit over time
- More powerful than just final profit — it shows the **shape** of how you make money

What a healthy equity curve looks like:

- Steady upward trend = consistent edge
- Small dips (drawdowns) that recover = normal and expected
- No single week that makes or breaks everything = not luck dependent

Red flags:

- Flat for months then one lucky week = got lucky, not real edge
- Sharp spike then crash = model overfit, broke when conditions changed

This is exactly what hedge funds use to evaluate strategies before deploying real capital.

---

xG — Expected Goals

Definition

xG = the probability that a specific shot results in a goal, based on historical data about similar shots.

Every shot gets a value between 0 and 1:

- Penalty = ~0.76 (high probability)
- Header from 30 yards = ~0.02 (very low)
- One-on-one with keeper = ~0.45

Why xG > Actual Goals for Prediction

- Actual goals include luck — deflections, keeper errors, posts
- xG measures **underlying quality of chances created**
- Over a season, teams trend toward their xG
- Lucky teams regress down. Unlucky teams improve.
- **Goals tell you what happened. xG tells you what should have happened.**

In Your Project

```sql
home_xg NUMERIC,
away_xg NUMERIC,
home_roll_xg_for_5 NUMERIC,  -- rolling avg xG last 5 games
away_roll_xg_for_5 NUMERIC,
```

- Current model uses actual goals — starting point
- Next version should use rolling xG features for better accuracy
- xG data source: **Understat.com** — free, covers top European leagues

---

SQL Concepts Used in This Project

CTE (Common Table Expression)

```sql
```

```
WITH latest AS (
  SELECT ...  -- solve one problem first
)
SELECT ...
FROM latest   -- use it like a table
```

- A **named temporary result** you can reference like a table
- Like a variable in Python: `latest = calculate(); use(latest)`
- Benefits: readability, reusability, thinking in steps
- Without CTE = one giant nested query that's impossible to debug

DISTINCT ON (PostgreSQL specific)

```sql
SELECT DISTINCT ON (p.match_id, p.market, p.model_version)
  p.*
FROM predictions p
ORDER BY p.match_id, p.market, p.model_version, p.created_at DESC
```

- For each unique combination of columns, keep **ONE row only**
- `ORDER BY created_at DESC` = keep the most recent one
- Used here because predictions are regenerated as odds change — you only want the latest per match/market

Normalisation

- Store data **once**, reference it everywhere else by ID
- BAD: storing "Manchester United" in 500 match rows
- GOOD: `id=1, name="Manchester United"` in teams table, matches store `home_team_id=1`
- Benefits: change name once → reflected everywhere, no spelling errors, no duplicates
- **Single source of truth**

JOIN Types

```sql
INNER JOIN  -- only rows matching on BOTH sides
LEFT JOIN   -- ALL rows from left, NULL if no match on right
RIGHT JOIN  -- all rows from right, NULL if no match on left
```

Scenario	Use
Must have odds to appear	INNER JOIN
Show fixture even without odds yet	LEFT JOIN

Your fixtures route uses LEFT JOIN for odds:

```sql
LEFT JOIN odds_markets om ON om.match_id = m.id
```

- With INNER JOIN: fixtures with no odds disappear — app looks broken
- With LEFT JOIN: all fixtures show, odds columns are NULL until data arrives
- LEFT JOIN = "show me everything, bring odds along IF they exist"

JOIN Aliases

```sql
JOIN teams ht ON m.home_team_id = ht.id   -- ht = home team
JOIN teams at ON m.away_team_id = at.id   -- at = away team
```

- Same table joined twice needs two different names
- Aliases make it readable and unambiguous

---

async/await — Non-Blocking Code

Sync vs Async

```python
# Sync (blocking) — waits for each one
data1 = fetch_season(2019)  # wait...
data2 = fetch_season(2020)  # then wait...

# Async (non-blocking) — runs together
data1, data2 = await asyncio.gather(
    fetch_season(2019),
    fetch_season(2020)
)
```

- `async def` = this function knows how to pause and resume

- `await` = pause here, but let other things run meanwhile
- If a function uses `await`, it must be `async def`
- `asyncio.run(main())` = entry point that runs async code

Why it matters for your seeder

Fetching 6 seasons of data potentially at the same time instead of one by one — much faster. Required by the `understat` library.

ETL Pipeline — Extract Transform Load

Your data seeder is an ETL pipeline:

- **Extract** = pull data from Understat and API-Football
- **Transform** = clean and reshape to match your schema
- **Load** = insert into PostgreSQL

This is one of the most important concepts in data engineering.

The Full Lambda Calculation Journey

Lambda comes from `estimate_lambdas()`. Full trace for Liverpool vs Fulham:

Step 1 — Query Liverpool's last 5 home matches (no leakage):

```sql
WHERE home_team_id = Liverpool
AND kickoff_utc < match_kickoff  -- never sees future data
ORDER BY kickoff_utc DESC LIMIT 5
```

Step 2 — Calculate averages:

```
Goals scored:   avg_scored   (attack strength)
Goals conceded: avg_conceded (defensive weakness)
```

Step 3 — Combine attack vs defence:

```python
lam_home = (home_scored + away_conceded) / 2
# Liverpool attack + Fulham defensive weakness
```

Step 4 — Shrink toward league baseline:

```python
lam_home = (lam_home + league_avg_home) / 2
# Protects against small sample sizes
```

Step 5 — Clamp:

```python
lam_home = max(0.2, min(lam_home, 3.5))
```

Full data journey:

```
Understat → DB → last 5 matches → avg scored/conceded
→ combine attack + defence → shrink → clamp
→ lambda → Poisson → probability → edge/EV → paper bet
```

Poisson Maths — Deep Dive

PMF Formula

```
P(X = k) = (e^(-λ) × λ^k) / k!
```

Each part:

- `e^(-λ)` — base rate, shrinks as lambda grows
- `λ^k` — grows as k increases
- `k!` — removes overcounting (order doesn't matter)

Example — λ=1.5, P(score exactly 2):

```
e^(-1.5) = 0.2231
1.5^2    = 2.25
2!       = 2
P(X=2)   = 0.2231 × 2.25 / 2 = 0.2510 = 25.1%
```

CDF = sum of PMF values

```
P(X ≤ 2) = P(X=0) + P(X=1) + P(X=2)
```

prob_over_total_goals explained

```python
lam_total = lam_home + lam_away  # independent Poisson variables add
k = int(math.floor(line))        # floor(2.5) = 2
return 1.0 - poisson_cdf(k, lam_total)  # complement rule
```

Why lambdas add: Home and away goals are independent. Key Poisson property: independent Poisson variables sum → their lambdas sum.

Why floor: Betting lines end in .5. Over 2.5 = 3 or more. Floor gives last whole number under the line.

Visual:

```
Goals:  0    1    2  |  3    4    5+
        ← Under 2.5 →|← Over 2.5 →
        CDF(2)       | 1 - CDF(2)
```

---

xG — Current Status in Your App

Where xG is stored: `matches.home_xg` and `matches.away_xg` — populated by Understat seeder for all 2581 matches.

Where model currently reads: `home_goals` and `away_goals` — NOT xG yet. This is Version 1.

Why xG alone is tricky:

- Teams consistently over/underperform xG (finishing quality)
- Goalkeeper quality not captured
- Sample size issues
- Understat's xG model isn't perfect

Model upgrade path:

Version	Data	Notes
v1 (current)	Goals only	Working, baseline
v2	xG only	Better but noisy
v3	Hybrid (0.4 goals + 0.6 xG)	Best approach
v4	Hybrid + home advantage + rest days	Professional

To upgrade to v2: Change `home_goals` → `home_xg` in `estimate_team_avgs_last_n()` query.

Rate Limiting

```python
import time
time.sleep(6)  # after each API call
```

- football-data.org free tier = 10 requests per minute
- 60 seconds ÷ 10 requests = 6 seconds per request
- `time.sleep(6)` spaces requests to never exceed the limit
- Without it: 429 error (too many requests) and script stops

Data Sources Summary

Source	What it provides	Used in
Understat	Historical matches + xG	seed_historical.py
football-data.org	Upcoming fixtures	seed_fixtures.py
The Odds API	Current bookmaker odds	seed_fixtures.py

Known limitations:

- The Odds API free tier = no historical odds (backtesting uses model accuracy not profitability)
- Understat library requires aiohttp + TCPConnector fix for Windows DNS
- football-data.org free tier = current season only

Still To Learn (Coming Up)

- [ ] update_results.py — marking finished matches
- [ ] Simple frontend to display predictions
- [ ] README